Operating Systems INSY 8223 Group 3 Assignment Report (Enhanced Version)

Introduction

This report covers concepts from Threads, Deadlock, and Memory Management. Each answer includes explanation, reasoning, and where applicable, diagrams or structured representations to demonstrate deep understanding.

1. Circular Wait Condition

Circular wait is one of the four necessary conditions for deadlock. It occurs when a set of processes are waiting for each other in a circular chain.

```
P1 -> R2 -> P2 -> R3 -> P3 -> R1 -> P1
```

In this cycle, no process can proceed, causing deadlock.

2. Banker's Algorithm

The Banker's Algorithm is a deadlock avoidance technique. It checks whether granting a resource request keeps the system in a safe state.

A safe state means all processes can complete execution without entering deadlock.

3. User-Level vs Kernel-Level Threads

User-level threads are managed by applications and are faster. Kernel-level threads are managed by the OS and allow true parallel execution.

User threads have low overhead, while kernel threads provide better system control and concurrency.

4. pthread_join()

pthread_join() blocks the calling thread until the specified thread finishes execution.

```
pthread_join(thread, NULL);
```

This ensures synchronization and prevents premature program termination.

5. Benefits of Multithreading

1. Responsiveness – UI remains active during tasks. 2. Resource Sharing – Threads share memory. 3. Parallelism – Multiple cores execute tasks simultaneously.

6. Static vs Dynamic Loading

Static loading loads the entire program into memory at once. Dynamic loading loads modules only when needed.

Dynamic loading is preferred for large applications or limited memory systems.

7. External Fragmentation

External fragmentation occurs when free memory is scattered into small blocks.

```
[Process][Free][Process][Free][Free][Process]
```

Compaction rearranges memory to create larger contiguous blocks.

Cost: High CPU usage and execution time.

8. Single vs Multiple Partition Allocation

Single partition allows only one process in memory. Multiple partition divides memory into segments for multiple processes.

Multiple partition allocation is more flexible and efficient.

9. Page Fault

A page fault occurs when a process accesses a page not in memory.

Steps: 1. Detect fault 2. Locate page on disk 3. Load into RAM 4. Update page table 5. Resume execution

10. Segmentation vs Paging

Segmentation divides memory based on logical units (code, stack, data) and is visible to the user.

Paging divides memory into fixed-size blocks and is managed internally by the OS.